



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

ELIXIR PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of Elixir?

Threat Model

Q6 How does Elixir implement object-oriented or functional programming?

Observability

Q2 How does Elixir handle type checking: static or dynamic?

Architecture

Q7 How does Elixir handle errors?

Lifecycle

Q3 How does Elixir manage memory?

Configuration

Q8 What testing tools are commonly used in Elixir?

Compliance

Q4 What is Elixir's concurrency model?

Hardening

Q9 What makes Elixir well-suited for its target domain?

Testing

Q5 How are packages and dependencies managed in Elixir?

SSO / IdP

Q10 What are common performance pitfalls in Elixir?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Elixir was designed with specific priorities around

Mitigation

Elixir was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A6 Elixir supports classes and inheritance for OOP,

Observability

Elixir supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern Elixir programs blend both paradigms depending on the problem.

A2 Elixir uses its type system to catch

Primitives

Elixir uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A7 Elixir surfaces errors via exceptions, Result/Either types,

Information

Elixir surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A3 Elixir uses one of three approaches: manual

Hardening

Elixir uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A8 Elixir has a standard or widely adopted

Governance

Elixir has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A4 Elixir supports concurrency through threads, coroutines,

Gotchas

Elixir supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A9 Elixir excels in its domain due to

Assessment

Elixir excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A5 Elixir uses a package manager and a

Federation

Elixir uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A10 Typical performance issues in Elixir include excessive

Optimization

Typical performance issues in Elixir include excessive allocations, blocking the main thread or event loop, $O(n^2)$ data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.